PART 5

The Claude API in Depth

Five chapters on the API as an API: who you are and which client builds the request, every parameter in the body, the response and its stop reasons, what to do when a call fails, and the batch, cache and cost controls that decide what the whole thing costs.

IN THIS PART

- 26. Authentication, SDKs and REST
- 27. Request Parameters and Sampling Controls
- 28. Response Handling and Stop Reasons
- 29. Errors, Retries, Timeouts and Backoff
- 30. Batch Processing, Prompt Caching and Cost Controls

CHAPTER 26

Authentication, SDKs and REST

In this chapter

Introduction: Four Hours Spent Debugging an Empty String . . . 4
26.1 The Front Door: What Is Actually on the Wire . . . 6
26.2 Two Credentials, and the Chain That Chooses Between Them . . . 7
26.3 What a Key Scopes, and What It Does Not . . . 8
26.4 Expiration, Rotation and the Audit Trail . . . 9
26.5 Workload Identity Federation . . . 10
26.6 anthropic-version: What Versioning Promises . . . 11
26.7 anthropic-beta: A Comma-Separated List With Dates In It . . . 11
26.8 The Official SDKs: Seven Languages and What They Add . . . 13
26.9 SDK Middleware: The Documented Extension Point . . . 14
26.10 The ant CLI . . . 14
26.11 Raw REST: What You Give Up . . . 16
26.12 Five Platforms, Five Front Doors . . . 16
26.13 Errors at the HTTP Level . . . 17
26.14 Visual Analogies . . . 18
26.15 Architecture: Where the Front Door Sits . . . 19
26.16 Examples . . . 20
26.17 Code Walkthrough: The Resolver That Counts Presence . . . 21
26.18 Mini Project: The Front-Door Harness . . . 22
26.19 Panel A — Eleven Environments, Two Resolvers . . . 23
26.20 Panel B — Twenty Rejections, No API Key . . . 24
26.21 Panel C — One Request, Three Clients, 202 Bytes . . . 25
26.22 Panel D — The Defaults You Did Not Set . . . 26
26.23 Panel E — Six Leaks, Two Scanners, Eighteen Operations . . . 27
26.24 Panel F — Forty-Five Cells, Four of Them Unknown . . . 29
26.25 Evaluation and Deployment . . . 29
26.26 Hands-on Labs . . . 30
26.27 Lab Solutions . . . 30
26.28 Best Practices . . . 31

26.29 Common Mistakes 33
26.30 Performance Considerations 35
26.31 Security Considerations 36
26.32 Real Production Examples 36
26.33 Certification Notes 37
26.34 Twenty Practice Questions 38
26.35 Answers and Explanations 39
26.36 Ten Practical Exercises 40
26.37 Five Challenge Labs 40
26.38 Ten Interview Questions 41
26.39 Summary 41
26.40 Glossary 43
26.41 Further Reading 44

Part 5 of 13 · Chapter 26 of 64. Next: Chapter 27 — Request Parameters and Sampling Controls.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Send an authenticated Claude API request with nothing but an HTTP client, naming every header and saying what each one is for.
2. Choose between a static API key and a Workload Identity Federation bearer token, and configure key expiration, rotation and scope deliberately rather than by default.
3. Trace the documented **credential precedence chain** and predict which credential a client will pick from an environment — including the case where a variable set to the empty string shadows everything below it.
4. Say exactly what `anthropic-version: 2023-06-01` promises, and what it explicitly permits Anthropic to change underneath you.
5. Assemble an `anthropic-beta` list correctly, including endpoint-scoped headers, mutually exclusive pairs, and why the date is part of the name.
6. Compare the seven official SDKs, the `ant` CLI and raw REST, and say for each what it adds over the wire and what it changes without being asked.
7. Register SDK middleware, reason about its ordering, and identify the built-in refusal-fallback middleware from Chapter 25 as one instance of the pattern.
8. Map authentication, model IDs, request-size ceilings and prompt-cache scope across the five platforms, marking anything unverified as UNKNOWN rather than assuming it matches.

NOTE

Prerequisites

Chapter 14 for model IDs, the three ID formats and the five platforms, and for the rule that a fact is verified on a date or it is UNKNOWN. **Chapter 16** for the Messages API itself — the endpoint, the three headers, the required fields and `stop_reason` as a control-flow branch — and for `ch16/wire.py`, which this chapter extends. **Chapter 17** for the Files API beta header and for fixture provenance. **Chapter 19** for the organisation/workspace/key hierarchy, the RPM bucket and `retry-after`. **Chapter 23** for the SDKs silently transforming a JSON Schema, which is the first instance of this chapter's central failure. **Chapter 25** for the refusal-fallback middleware and the exact-date discipline on beta headers.

This chapter opens Part 5. **Request parameters and sampling controls** are **Chapter 27**; **response handling and stop reasons in depth** are **Chapter 28**; **errors, retries, timeouts and backoff as a discipline** are **Chapter 29**; **batch, caching and cost controls** are **Chapter 30**. Chapter 26 deliberately stops at the front door: who you are, which client builds the request, and which platform receives it.

Introduction: Four Hours Spent Debugging an Empty String

A payments company moved its Claude integration off static API keys and onto Workload Identity Federation. The engineering was careful. A service account, a federation issuer and a federation rule were created in the Console; a projected OIDC token was mounted into the pod; the four federation environment variables went into the deployment manifest. The change was reviewed by two people and shipped on a Thursday.

Every request failed with `401 authentication_error`.

The on-call engineer did what the error told her to do and spent four hours on the federation configuration. She decoded the projected JWT and compared `iss` against the registered issuer byte for byte. She checked `aud`, `sub`, `exp` and the thirty-second clock skew. She confirmed the JWKS was reachable over HTTPS on port 443. She read the authentication history page in the Console, which showed **no exchange attempts at all** — a fact she noted and then, understandably, discounted, because the absence of a log line looks like a logging problem.

The cause was one line in the manifest. The old `ANTHROPIC_API_KEY` entry had not been deleted; it had been set to `""`, which felt tidier than removing it and left the key name in the file as documentation. The SDK's documented credential precedence chain puts `ANTHROPIC_API_KEY` above federation, and the documentation is explicit about what "set" means:

> *"A variable that is set to an empty string still occupies its slot in the credential precedence chain. If `ANTHROPIC_API_KEY=""` is exported, the SDK selects the API-key path with an empty key rather than falling through to federation. Unset unused credential variables rather than blanking them."*

The client never attempted a token exchange. It authenticated as nobody, with an empty key, and the API said so correctly. There was no exchange log because there was no exchange.

Section 26.19 runs eleven environments through two resolvers — the documented one, keyed on **presence**, and the naive one everybody writes, keyed on **truthiness** — and they disagree on **four of eleven**. This one is among them. So is the mirror-image case, a blank key over an active profile, and so is a profile that is named but has never been created, which the documentation says is *an error, not a fall-through*, and which the naive resolver happily accepts.

That is the first of this chapter's four measured failures, and all four are the same failure wearing different clothes: **a convenience decided something on your behalf and did not tell you.**

The second is the one Chapter 23 already met. There, the SDKs silently rewrote a JSON Schema — stripping `minimum`, `maxLength` and unsupported formats into the description — so that the bytes on the wire were not the bytes you wrote. This chapter finds the same class of surprise three more times. Section 26.21 builds one logical request three ways — raw REST, an SDK-shaped call, and the `ant` CLI's flags-over-stdin merge — and the bodies serialise to **identical bytes, 202 of them**. That is the good news, and it is worth stating plainly: an SDK is a convenience, not the protocol. But the **headers** do not match, and every difference is something added or dropped for you. The SDK's `beta` namespace adds an endpoint-scoped `anthropic-beta` header you never wrote. The CLI, given two `--beta` flags, sends **only the first** — a documented behaviour, silent at runtime, identical body, different headers.

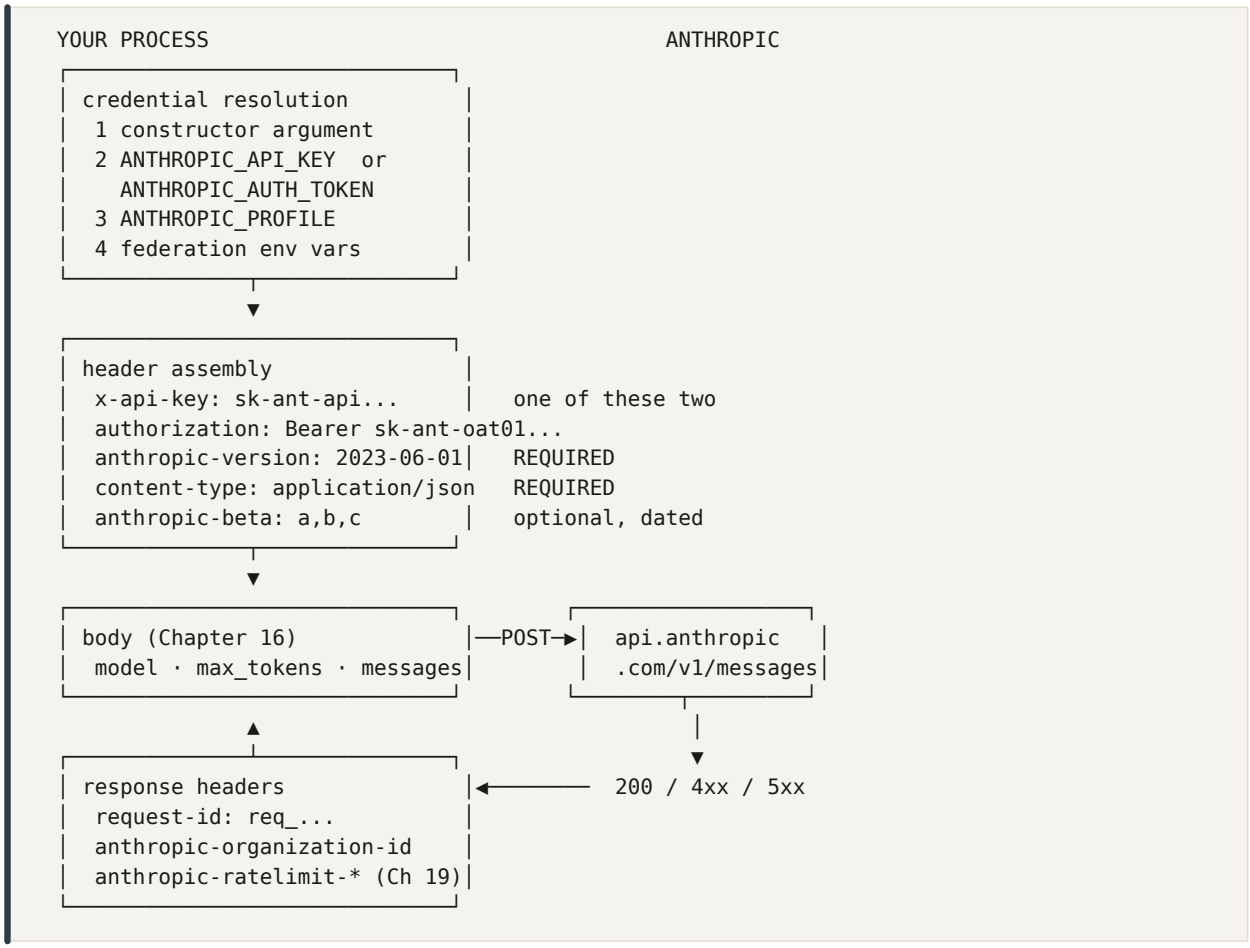
The third failure costs money. Nobody in your codebase wrote `max_retries=2` or `timeout=600`, because those are the constructor defaults you get by passing nothing. Section 26.22 shows what they mean. Three attempts at a ten-minute timeout is **1,800 seconds — thirty minutes — before the caller sees an error**, on a client whose timeout is documented as ten. And the retries draw on Chapter 19's RPM bucket: a plan running at **4,800 requests per minute against a 5,000 RPM Build-tier limit is at 96.00 per cent of the bucket**, comfortable, until a five per cent transient failure rate turns it into **5,052 RPM, or 101.04 per cent**, and the bucket starts returning 429s that the same default retries then retry. At a thirty per cent failure rate — an overload episode — it is **6,672 RPM, 133.44 per cent**. Not one line of application code changed.

The fourth is about secrets. Section 26.23 scans ten artefacts for leaked credentials. Six contain a real one. The naive pattern — the `sk-ant-api` prefix, which is the one a working developer sees every day — finds **four of six**, with no false positives. The full pattern finds **six of six**. The two it misses are an Admin API key in a committed notebook output and an OAuth access token in a Terraform state file, and those are precisely the two credentials with the largest blast radius: across eighteen documented operations, a workspace key reaches **six**, an Admin API key reaches **nine** administrative ones including deactivating every other key in the organisation, and an `org:admin` OAuth token reaches **eleven**. The scanner most people write is blind to exactly the credentials that matter most.

Part 5 works through the Claude API from the outside in. Chapter 16 taught the request. This chapter teaches everything around it.

26.1 The Front Door: What Is Actually on the Wire

Strip away every client and a Claude API call is an HTTP POST to a documented URL carrying four things: a credential, a version, a content type and a JSON body. Chapter 16 covered the body. Everything else is this chapter.



Two response headers appear on **every** response and are worth wiring into your logging before anything else. `request-id` carries a value such as `req_018EeWyXxfu5pfWkrYcMdjWG` and is the identifier Anthropic support asks for; the same value appears as the `request_id` field inside error bodies. `anthropic-organization-id` names the organisation the credential belongs to, which is the cheapest

possible answer to "which account did that request bill?"

The rate limit headers are Chapter 19's material and are not repeated here.

> DEFINITION
>
> **Definition — credential, version, beta**
>
> A **credential** identifies the caller. The API accepts exactly two forms: a static API key in `x-api-key`, or a short-lived bearer token in `Authorization`. A **version** is the `anthropic-version` header, which pins the request and response *shape*. A **beta** is an entry in the comma-separated `anthropic-beta` header that opts one request into a pre-GA feature.

26.2 Two Credentials, and the Chain That Chooses Between Them

The documentation names two authentication methods, and both grant *the same access* to Claude API endpoints. They differ in how the secret is managed, not in what it can do.

	API key	Workload Identity Federation
Credential	static `sk-ant-api...` string	short-lived token exchanged from an IdP JWT
Header	`x-api-key`	`Authorization: Bearer <token>`
Minted by	you, in the Console	`POST /v1/oauth/token`, by the SDK
Lifetime	your chosen expiration, or never	60 s to 24 h, default 3,600 s
Best for	local development, prototyping, scripts, single-tenant servers where you control secret storage	production on AWS, Google Cloud or Azure; CI/CD; Kubernetes
Rotation	manual, or by expiry	automatic; the SDK refreshes before expiry

An API key is read from the `ANTHROPIC_API_KEY` environment variable by every SDK and by the CLI. Nothing else is required to make a first call.

The interesting part is not either method. It is what happens when both are configured, which in a real migration is *always*, because environments accumulate. The documented precedence chain is:

Order	Source	Note
1	Constructor argument (`api_key=`, `auth_token=`, `credentials=`)	Always overrides everything else.

Order	Source	Note
2	`ANTHROPIC_API_KEY` or `ANTHROPIC_AUTH_TOKEN`	*"Shadows federation entirely. Unset these when migrating from API keys."*
3	`ANTHROPIC_PROFILE`	Loads `<config_dir>/configs/<name>.json`. *"A missing named profile is an error, not a fall-through."*
4	Federation environment variables	Requires all of `ANTHROPIC_FEDERATION_RULE_ID`, `ANTHROPIC_ORGANIZATION_ID`, `ANTHROPIC_SERVICE_ACCOUNT_ID` and one of `ANTHROPIC_IDENTITY_TOKEN_FILE` / `ANTHROPIC_IDENTITY_TOKEN`.

Read rung 2 again with the empty-string rule in mind. The chain tests **presence**, not truthiness. `ANTHROPIC_API_KEY=""` is present. It wins.

> WARNING
>
> **The one-line difference that caused the incident**
>
> `if "ANTHROPIC_API_KEY" in env:` and `if env.get("ANTHROPIC_API_KEY"):` look like the same test. Against the documented chain they are not, and Section 26.19 measures the gap at four scenarios out of eleven. When you migrate away from a credential, **unset the variable — do not blank it**.

There is one further subtlety that is easy to miss. When a profile is loaded, environment variables fill fields the profile omits but **never override fields the profile sets explicitly**. So `ANTHROPIC_WORKSPACE_ID` populates `workspace_id` only when the active profile leaves it blank. This is the opposite of the usual "environment beats file" instinct.

The active profile name itself resolves in its own order: `$ANTHROPIC_PROFILE`, then the contents of `<config_dir>/active_config` (a one-line file written by `ant profile activate`), then the literal name `default`. The configuration directory is `$ANTHROPIC_CONFIG_DIR`, then `~/.config/anthropic` on Linux and macOS, then `%APPDATA%\Anthropic` on Windows. Claude Code and the Claude Agent SDK honour the same resolution order, which is convenient and occasionally surprising: a profile you configured for a service will also authenticate your editor.

26.3 What a Key Scopes, and What It Does Not

Chapter 19 established the hierarchy: an organisation contains workspaces, a workspace contains API keys, and **rate limits and spend caps live at the organisation level, so a key carries none of its own**. This section adds what a key *does* scope.

> *"API keys are scoped to a specific workspace. When you create an API key in a workspace, it can only access resources within that workspace."*

The resources that are workspace-scoped are named explicitly: **Files** created through the Files API, **Message Batches**, and **Skills**. That is a much sharper statement than it first appears. A `file_id` uploaded with your development key is not visible to your production key, and the failure mode is a `404 not_found_error` on a request that is otherwise perfect. Chapter 17 said a `file_id` fixes bytes, not

tokens; Chapter 26 adds that it also fixes *workspace*.

Some things a workspace key cannot manage at all:

- **The Admin API** — organisation members, invites, workspaces themselves, API keys — requires an Admin API key beginning `sk-ant-admin...`, or an `org:admin` OAuth bearer token.
- **The service-account, federation-issuer and federation-rule endpoints** accept **only** the `org:admin` OAuth token. An Admin API key is refused there. This is one of the few places where the more famous credential is the weaker one.
- **MCP tunnels** are managed with a `workspace:manage_tunnels` OAuth token obtained through federation, not with a workspace API key at all.

And one fact that belongs in a cost chapter but lands here because it is a property of the front door: **prompt caches are isolated per workspace** on the Claude API, Claude Platform on AWS and Microsoft Foundry — but **per organisation** on Amazon Bedrock and Google Cloud. Splitting a workload across two workspaces on the Claude API splits its cache, and Chapter 11 priced what that costs.

Other workspace facts worth holding: identifiers use the `wrkspc_` prefix; there is a maximum of **100 workspaces** per organisation, archived ones excluded; the **Default Workspace has no ID and does not appear in list endpoints**, which is why Chapter 19 found a null `workspace_id` in the Usage API; and **archiving a workspace immediately revokes every API key in it and cannot be undone**.

26.4 Expiration, Rotation and the Audit Trail

Since key expiration became a creation-time choice, "rotate your keys" stopped being purely a matter of discipline. When you create a key in the Console you choose an expiration: a preset of **3 hours, 1 day, 7 days or 30 days**, a custom duration, or **Never** for keys you intend to store in a secrets manager and rotate yourself. If your organisation has a maximum-expiration policy, the Console caps the presets and custom durations at the policy maximum and **Never becomes unavailable**.

Three properties matter operationally:

1. **Expiration is set at creation and cannot be changed afterwards.** There is no extending a key. Rotation means minting a new one.
2. **Anthropic emails the key's creator before expiry** — 7 days ahead for keys with a lifetime of at least 14 days, 1 day ahead for those of at least 7 days. Keys with shorter lifetimes **expire with no warning email at all**, which makes a 3-hour key a deliberate choice, not a safe default.
3. **An expired key returns** `401 authentication_error` **and cannot be reactivated.** You create a new one.

The audit trail is the Admin API. Both `List API Keys` and `Retrieve API Key` report an `expires_at` timestamp, `null` for keys without an expiration, so a scheduled job can enumerate every key in the organisation and alert on the ones about to lapse — or, more usefully, on the ones that never will. Chapter 19's Console → Usage → Export remains the per-key spend trail; `expires_at` is the per-key *lifetime* trail, and you want both.

BEST PRACTICE

Rotate on a schedule you can actually keep

An expiration you set and then forget becomes an outage on a date nobody remembers choosing. Either set **Never** and rotate from a secrets manager on a schedule your platform enforces, or set a duration **and** wire the `expires_at` field into the same alerting that watches your certificates. The failure mode of a half-adopted policy is worse than either policy alone.

26.5 Workload Identity Federation

Federation replaces the static secret with an exchange. The workload already has an identity — an AWS IAM role, a Google Cloud service account, a Kubernetes projected service-account token, a GitHub Actions OIDC token, a SPIFFE SVID, an Entra ID or Okta token — and that identity's JWT is exchanged at `POST /v1/oauth/token` for a short-lived Claude API access token prefixed `sk-ant-oat01-`. There is no `sk-ant-api...` string to mint, distribute or rotate.

Three resources are created in the Console: a **service account** (`svac_...`), a **federation issuer** (`fdis_...`) and a **federation rule** (`fdrl_...`). The exchange body uses the RFC 7523 `jwt-bearer` grant and carries `grant_type`, `assertion`, `federation_rule_id`, `organization_id`, `service_account_id`, and conditionally `workspace_id`. The response is a standard OAuth 2.0 token response: `access_token`, `token_type: "Bearer"`, `expires_in` and `scope`.

The scope table currently has one entry. `workspace:developer` grants *"all non-administrative Claude API endpoints in the rule's workspace"* — Messages including streaming and token counting, Models, Managed Agents and sessions, Files, and Skills — and the documentation says plainly that this *"matches the access an API key issued for the same workspace has."* A request outside the token's scope returns **403**. Finer-grained scopes, per resource or read-versus-write, **are not currently available**.

The validation rules are strict in ways that produce confusing failures if you have not read them:

Constraint	Rule
`token_lifetime_seconds`	60 to 86,400; default 3,600; outside the range is rejected at request time
Assertion size	at most **16 KiB**
Signing algorithm	asymmetric only — ES256/384, RS256/384, PS256/384. **HS256 and `none` are rejected**
Required claims	`sub` present; `exp` in the future
Clock skew	30-second leeway on `exp`, `nbf` and `iat`
Issuer URLs	scheme must be `https`, port must be 443, host must be a public DNS name resolving to public IPs
Rule `match` block	at least one of `subject_prefix`, `claims` or `condition`; a block containing only `audience` is **rejected**

That last rule is a guardrail against a mistake with no other symptom: a rule matching only on audience would accept **every** token from that issuer.

The documentation is also unusually candid about what federation does not buy you. It *"does not, on its own, guarantee end-to-end security: the trust chain is only as strong as your identity provider's configuration, and a long-lived secret one hop upstream (for example, a static cloud credential that can mint IdP tokens) can still undermine it."* Moving the secret is not the same as removing it.

One operational detail that produces a one-minute outage nobody can reproduce: Anthropic **caches the fetched JWKS**, so if your IdP publishes a new signing key and immediately starts signing with it, exchanges can fail for **up to a minute**. Publish new keys at least fifteen minutes before first use.

Finally, debugging. A `400 invalid_grant` is *intentionally opaque* — the specific cause is logged server-side only. The documented first step is the **authentication history page** in the Console, which shows the issuer and rule evaluated, the claims inspected, and which validation step failed. The engineer in this chapter's introduction was right to look at it; her mistake was in explaining away an empty page.

26.6 `anthropic-version`: What Versioning Promises

`anthropic-version` is required on every request, and there is currently one version you should send: `2023-06-01`. The header is not decoration. It is a contract, and the contract is worth quoting in full because half of it is a list of things that *will* change.

For a given version, Anthropic **preserves**:

- existing input parameters
- existing output parameters

Anthropic **may**:

- add additional optional inputs
- add additional values to the output
- change conditions for specific error types
- **add new variants to enum-like output values (for example, streaming event types)**

That fourth bullet is the one to internalise. It is the documented reason Chapter 16's `stop_reason` handler raises on an unknown value instead of falling through, and the documented reason Chapter 17's SSE parser must tolerate event types it has never seen. Versioning promises your existing fields will not vanish. It explicitly reserves the right to hand you a value your `match` statement has no arm for.

The version history is short: `2023-06-01` introduced the current SSE format — incremental completions, named events, and the removal of the `data: [DONE]` event — and removed the legacy `exception` and `truncated` response values. `2023-01-01` was the initial release. Previous versions *"are considered deprecated and may be unavailable for new users."*

Every SDK sends `anthropic-version: 2023-06-01` for you and lets you override it, with a warning attached: *"Overriding default headers may result in incorrect types and other unexpected or undefined behavior in the SDK."* There is no good reason to.

26.7 `anthropic-beta`: A Comma-Separated List With Dates In It

A beta header opts one request into a pre-GA feature. The mechanics are simple and the failure modes are not.

The header takes a comma-separated list: `anthropic-beta: feature1,feature2,feature3`. Every SDK exposes a `beta` namespace and a `betas` parameter that becomes the header. The CLI takes `--beta feature1,feature2` — **one flag, comma-separated** — and here is the first trap, documented in the official page itself:

> *"Avoid repeating the flag: currently only the first flag's value takes effect."*

So `--beta files-api-2025-04-14 --beta task-budgets-2026-03-13` sends only the Files beta. There is no error, no warning and no difference in the request body. Section 26.21 reproduces this and asserts the header difference explicitly, because a test is the only thing that will ever catch it.

Feature names carry a date, following the pattern `feature-name-YYYY-MM-DD`, and the guidance is to *"always use the exact beta feature name as documented."* Chapter 25 measured what "exact" means: `server-side-fallback-2026-06-01` must carry that date and no other. A beta name with the wrong date is simply an unknown beta name, and an unknown beta name — or one your organisation does not have access to — returns a 400:

```
{
  "type": "error",
  "error": {
    "type": "invalid_request_error",
    "message": "Unexpected value(s) `invalid-beta-name` for the
      `anthropic-beta` header. Please consult our documentation at
      platform.claude.com/docs or try again without the header."
  },
  "request_id": "req_011CcnGfC9fELffo2EALu4Wd"
}
```

Some betas are **endpoint-scoped**: they must be present on every request to certain endpoints, regardless of what features you are using.

Endpoints	Required beta header
`/v1/agents`, `/v1/sessions`, `/v1/environments`	`managed-agents-2026-04-01`
`/v1/tunnels`	`mcp-tunnels-2026-06-22`
`/v1/memory_stores` and sub-resources	`agent-memory-2026-07-22`

The SDKs' `beta` namespaces add these automatically. *"Add them yourself only when making raw HTTP requests."* This is a genuine asymmetry between clients, and the reason a cURL command copied from a working SDK integration fails with a 400 that mentions nothing about SDKs.

And two endpoint-scoped betas are not always combinable. On memory-store endpoints `agent-memory-2026-07-22` **replaces** `managed-agents-2026-04-01`**, and sending both on the same request returns a 400.** The SDKs pick the correct one per endpoint.

Beta features are experimental and may *"have breaking changes with notice, be deprecated or removed, have different rate limits or pricing, or not be available in all regions."* That third clause is the one to price: a beta can cost differently from GA, and Chapter 30 returns to it.

26.8 The Official SDKs: Seven Languages and What They Add

There are seven official client SDKs — **Python, TypeScript, C#, Go, Java, PHP and Ruby** — plus the `ant` CLI, plus two libraries that are *not* general-purpose Messages clients but expose Claude through another framework's surface: the Apple Foundation Models Swift package and an OpenAI SDK compatibility layer.

The documented benefits are worth listing precisely, because each one corresponds to a chapter of this book:

The SDK does	Which chapter it replaces work from
Automatic header management (`x-api-key`, `anthropic-version`, `content-type`)	this chapter, §26.1
Type-safe request and response handling	Chapter 16's typed content blocks
Built-in retry logic honouring `retry-after`	Chapter 19's pacing
Streaming support and accumulation helpers	Chapter 17's SSE reassembler
Request timeouts and connection management	§26.22 below
Auto-paginating iterators	§26.11
Typed exception hierarchies	Chapter 29
Schema transformation for structured outputs	Chapter 23 — *and this one changes the bytes*
The refusal-fallback middleware	Chapter 25

Language-level differences that actually matter in an architecture review:

- **Python** ships sync and async clients, Pydantic response models, an `aiohttp` backend for higher async concurrency, a `@beta_tool` decorator that generates a tool schema from a function signature and docstring, and a `tool_runner` that executes the agent loop for you. Requires Python 3.9 or later.
- **TypeScript** targets Node.js, Deno, Bun **and the browser** — which is a capability and a liability; see §26.31.
- **C#** targets .NET Standard 2.0+ and integrates with `IChatClient`.
- **Go** uses context-based cancellation and functional options, and surfaces every error as a single `*anthropic.Error` value you branch on by `StatusCode` rather than a class hierarchy.
- **Java** uses a builder pattern and `CompletableFuture` for async.
- **PHP** uses value objects and a builder pattern; **Ruby** uses Sorbet types and streaming helpers.
- **Request IDs**: Python and TypeScript expose `_request_id` directly on response objects. C#, Go, Java and PHP expose it through raw-response accessors.

Chapter 25 recorded that the refusal-fallback middleware ships in TypeScript, Python, Go, Java and C# — **not Ruby or PHP**. That is the pattern to expect: the surface is the same everywhere, the conveniences are not.

WARNING

Two SDK defaults you did not choose

`max_retries` defaults to **2** — up to three attempts — and `timeout` defaults to **10 minutes**. Both are documented, neither appears in your code, and together they mean a "ten-minute" client can take **thirty minutes** to report failure. Section 26.22 prices them.

There is one more guard worth knowing because it fires at runtime rather than compile time: the SDKs *"throw a* `ValueError` *if a non-streaming request is expected to take longer than approximately 10 minutes"*, and passing `stream=True` or overriding `timeout` disables the check. Chapter 17 met the sibling of this rule — the 21,333-token `max_tokens` streaming requirement — and both are client-side guards, not API limits.

26.9 SDK Middleware: The Documented Extension Point

Every SDK provides a middleware (or interceptor) hook that runs code before a request is sent and after the response returns. Each middleware receives the outgoing request and a `next` callable; anything before the `next` call runs on the way out, anything after runs on the way back. Documented uses are *"logging, custom retries, request annotation, and refusal fallback handling."*

Ordering is worth memorising because it is the opposite of what the word "order" suggests to some people:

- Middleware apply **in the order given**: the first middleware's "before" code runs **first**, and its "after" code runs **last**. It is an onion, not a queue.
- **Middleware registered on the client runs before middleware passed as a per-request option.**
- In **Go**, repeated `option.WithMiddleware` calls concatenate — client first, then method. In the other SDKs you pass an array, and **later entries wrap inner**.

Separately, each SDK accepts a **custom HTTP client** for proxies, TLS configuration or connection pooling. Only one is used per client; setting it **replaces** the default. The custom client receives requests **after all middleware has run**. In Python the documentation is specific about how to do this without losing the defaults: use `DefaultHttpxClient` and `DefaultAsyncHttpxClient` rather than raw `httpx.Client`, *"to ensure the SDK's default configuration (timeouts, connection limits, etc.) is preserved."*

The built-in refusal-fallback middleware from Chapter 25 is simply an instance of this pattern, and knowing that reframes it: the fallback behaviour is not magic in the client, it is a middleware you could have written, sitting in a chain you can inspect.

TIP

Middleware is where cross-cutting concerns belong

Request ID logging, correlation headers, cost accounting per call site, redacting prompt content before it reaches your logs, and a circuit breaker all belong in middleware rather than scattered through call sites. One place to read, one place to test, and — importantly for this chapter — one place that can print the exact bytes going out.

26.10 The `ant` CLI

`ant` is the official command-line client. It matters here for three reasons: it is the fastest way to see the wire, it is a legitimate production tool for scripting, and its input semantics are a compact illustration of this chapter's theme.

Commands follow a `resource action` pattern, with nested resources separated by colons: `ant <resource>[:<subresource>] <action> [flags]`. Beta resources live under a `beta:` prefix, and **commands in that namespace send the appropriate `anthropic-beta` header automatically** — `--beta` is only for overriding the default.

The global flags that earn their place:

Flag	What it does
`--format`	`auto`, `json`, `jsonl`, `yaml`, `pretty`, `raw`, `explore`
`--transform`	reshape the response with a GJSON path; on list endpoints it runs **per item**, not on the envelope
`-r`, `--raw-output`	print string results without quotes, like `jq -r`
`--profile`	pick a named profile for this invocation
`--base-url`	point at a different API host
`--debug`	print the **full HTTP request and response to stderr**, with API keys redacted
`--format-error`, `--transform-error`	the same two, applied to error responses

`--raw-output` and `--format raw` are different things and the documentation says so: `--raw-output` strips JSON quotes from string results; `--format raw` prints the response body's raw bytes **without auto-paginating**, and applies `--transform` to the pagination envelope rather than to each item.

Request bodies arrive three ways, and the merge rule is the interesting one. **Flags** carry scalars and short structured values in a relaxed YAML-like syntax. **Stdin** carries a full JSON or YAML document. **`@file` references** inline a file's contents into a string or binary field, with binary auto-encoded to base64 (`@file://` forces plain text, `@data://` forces base64, and a literal leading `@` is escaped with a backslash). And:

> *"Fields from stdin are merged with flags, with flags taking precedence."*

Silently. A heredoc that specifies `model: claude-sonnet-5` and a stray `--model claude-opus-5` on the command line produce a request for Opus with no indication that the document was overruled. Section 26.21 measures this as a body-bytes difference, which is exactly what it is.

`--debug` is the single most useful thing in this chapter for learning:

```
ant --debug beta:agents list
```

```
GET /v1/agents?beta=true HTTP/1.1
Host: api.anthropic.com
```

```
Anthropic-Beta: managed-agents-2026-04-01
Anthropic-Version: 2023-06-01
X-Api-Key: <REDACTED>
```

There is the endpoint-scoped beta header, added for you, printed where you can see it. Note also that the key is redacted — a small thing, and Section 26.23 counts it as the reason a `--debug` transcript in a CI log is **not** a leak.

Authentication has its own conveniences. `ant auth login` runs a browser-based OAuth flow and stores credentials under `$ANTHROPIC_CONFIG_DIR`, with `--no-browser` for remote hosts. The issued token is **scoped to one workspace**, chosen in the browser or with `--workspace-id`. To work across workspaces you log in under a named profile per workspace and switch with `ant profile activate`. `ant auth status` prints which credential source and workspace won — the command that would have ended this chapter's introduction in ninety seconds. And one rule that catches everybody:

> *"Profiles are only consulted when no API key is set. If* `ANTHROPIC_API_KEY` *is present in your environment, it overrides every profile."*

Which is rung 2 beating rung 3, exactly as the chain says.

26.11 Raw REST: What You Give Up

`ch16/wire.py` already proves that no SDK is required: it builds the body, parses the response and branches on `stop_reason` with nothing but the standard library. This chapter extends it with headers, credentials and a preflight. So what does going without cost you?

You give up, and must rebuild: typed exceptions and a retry policy (Chapter 29), `retry-after` honouring (Chapter 19), streaming reassembly (Chapter 17's `sse.py`), the schema transformation and client-side re-validation Chapter 23 documented, auto-pagination, the `_request_id` accessor, TCP keep-alive on long requests, and the endpoint-scoped beta headers of §26.7.

You gain: the bytes. Every one of the conveniences above is a place where the request you wrote and the request that was sent can differ, and §26.21 finds three such places without trying hard.

The honest recommendation is not "avoid the SDK". It is: **use the SDK, and own a way to print the bytes.** Middleware (§26.9) gives you that in one place, and `ant --debug` gives it to you at a shell prompt.

Pagination deserves a note since it is pure boilerplate you would otherwise write. Most newer list endpoints use a `page` / `next_page` cursor scheme with a `limit` query parameter, and some support `prev_page` — currently only `GET /v1/sessions`; on endpoints without backward pagination the field is **absent rather than null**. Older endpoints — Message Batches, Files, Models and several Admin endpoints — use `after_id` / `before_id` and return `has_more`, `first_id` and `last_id` instead. Every SDK provides an auto-paginating iterator, and **SDK auto-pagination is forward-only**; to go back you read `prev_page` yourself.

26.12 Five Platforms, Five Front Doors

Chapter 14 recorded that the same model wears three ID formats across five platforms. The front door differs in more than the name.

	Claude API	Claude Platform on AWS	Amazon Bedrock	Google Cloud	Microsoft Foundry
Operator	Anthropic	Anthropic, on AWS	AWS (partner)	Google (partner)	Anthropic, on Azure
Credential	API key or WIF bearer	AWS credentials, SigV4	AWS IAM	Google Cloud IAM	Azure
Model ID	`claude-opus-5`	Claude API format	`anthropic.claude-opus-5`	`claude-opus-5@20260609`	UNKNOWN
Max request	32 MB	32 MB	**20 MB**	**30 MB**	UNKNOWN
Prompt cache scope	workspace	workspace	**organisation**	**organisation**	workspace
Admin API	full	workspaces only	–	–	–
Server-side fallback	yes	yes	no	no	no
Fallback credit	yes	yes	yes	yes	yes

Four of forty-five cells in Section 26.24's machine-readable version are **UNKNOWN**, and they raise rather than defaulting — the Chapter 14 rule, restated because the temptation to assume "probably the same" is strongest precisely where platforms look interchangeable.

Two rows are load-bearing. The **request-size row** means a 25 MB payload that works on the Claude API is a `413 request_too_large` on Bedrock; Section 26.24 measures a 25 MB request as accepted by three platforms, refused by one and unverifiable on one. The **cache-scope row** is a cost difference of the kind Chapter 11 priced.

Two platform-specific operational facts are worth carrying:

- On **Claude Platform on AWS** every response carries **two** request IDs: `x-amzn-requestid` (primary, indexed in CloudTrail) and `request-id` (secondary). Use the AWS one for CloudTrail, the Anthropic one for support tickets. A 401 there *"can also indicate a problem with your AWS credentials or SigV4 signature"* rather than an API key. And if every request returns *"Outbound web identity federation is disabled for your account"*, the fix is `aws iam enable-outbound-web-identity-federation`, run **once per AWS account**.
- On the direct Claude API, a **413 is returned by Cloudflare before the request reaches the API servers**, so it will not appear in any Anthropic-side per-request telemetry you may be reconciling against.

26.13 Errors at the HTTP Level

Chapter 29 handles errors, retries and backoff as a discipline. This section gives the shape, because you cannot debug a front door without it.

Status	Type	Means
400	`invalid_request_error`	format or content of the request; also used for other unlisted 4xx
401	`authentication_error`	key malformed, revoked or expired; on Claude Platform on AWS, an AWS credential or SigV4 problem
402	`billing_error`	billing or payment information
403	`permission_error`	the key lacks permission for that resource
404	`not_found_error`	check the path and any resource IDs
409	`conflict_error`	concurrent modification, or a uniqueness violation
413	`request_too_large`	over the per-endpoint byte ceiling
429	`rate_limit_error`	Chapter 19
500	`api_error`	retry with exponential backoff; quote the request ID
504	`timeout_error`	consider streaming
529	`overloaded_error`	temporarily overloaded, across all users

Every error body has the same shape — a top-level `error` object with `type` and `message`, plus a sibling `request_id`:

```
{
  "type": "error",
  "error": {
    "type": "not_found_error",
    "message": "The requested resource could not be found."
  },
  "request_id": "req_011CSHoEeqs5C35K2UUqR7Fy"
}
```

Two disciplines follow. First, *"in accordance with the versioning policy, the values within these objects may expand, and it is possible that the* `type` *values will grow over time"* — so treat the error `type` the way Chapter 16 treats `stop_reason`: as an enum that will grow. Second, *"catch the SDK's typed classes rather than string-matching error messages, handling the most specific classes first."* The class names differ by language — `anthropic.NotFoundError` in Python, `Anthropic::Errors::NotFoundError` in Ruby, `com.anthropic.errors.NotFoundException` in Java, and a single `*anthropic.Error` you branch on by `StatusCode` in Go — which is itself an argument for putting the mapping in one place.

And the one that Chapter 17 already taught: **an error can arrive inside an HTTP 200** when streaming. None of this table applies to it.

26.14 Visual Analogies

VISUAL ANALOGY

The building pass and the visitor badge

An API key is a building pass with your name printed on it: it works until someone deactivates it, and if you drop it in the street whoever picks it up is you. A federation token is a visitor badge printed at reception from an ID you already carry: it expires this afternoon, it names the floor you may visit, and losing it costs you an afternoon rather than a career.

Where it breaks. Reception is only as good as the ID it trusts. The documentation makes exactly this point — a static cloud credential upstream that can mint IdP tokens undermines the whole chain. And unlike a real badge, an expired token is refreshed automatically by the SDK, so the *"it expires"* reassurance protects you against theft, not against misconfiguration.

VISUAL ANALOGY

A helpful assistant who tidies your outgoing post

The SDK is an assistant who addresses your envelopes, adds the stamp, keeps a copy, and posts it again if it comes back marked "try later". Marvellous, until you discover that the assistant has also been correcting what they consider errors in the address, and that "posts it again" means three times over half an hour before telling you.

Where it breaks. A real assistant would mention it. Chapter 23's schema transformation, this chapter's auto-added beta header, the discarded second `--beta` flag and the two default constructor values are four cases where nothing is mentioned. The assistant is not deceiving you; nobody wrote down which conveniences are silent, so the only way to know is to look at the post.

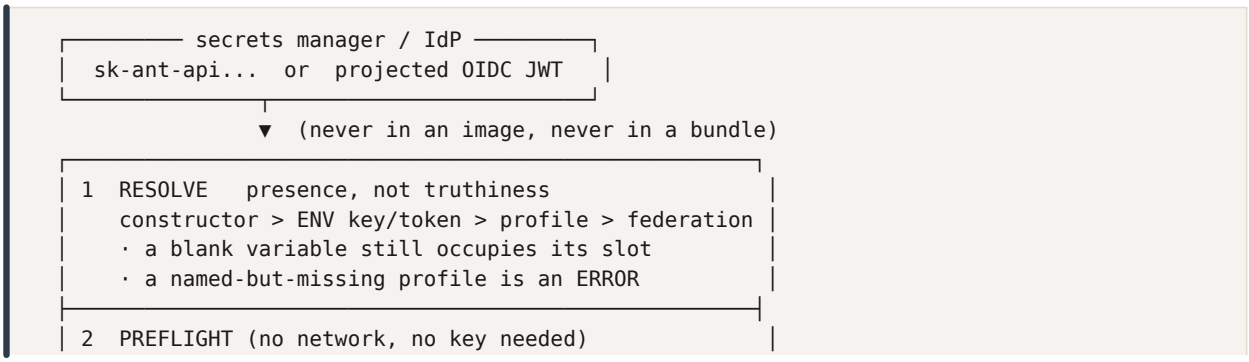
VISUAL ANALOGY

Master keys on a hotel key ring

Six of the artefacts in Section 26.23 leak a credential. Four are room keys — one workspace, one floor. One is the housekeeping master, and one is a temporary staff card. A scanner that only recognises room keys will confidently report finding all the room keys.

Where it breaks. A hotel key ring has a fixed number of key types. The credential prefixes here are documented, so the scanner *can* be complete, which means an incomplete one is a choice rather than a limitation. The analogy also understates the master: an Admin API key does not merely open the rooms, it can **change who holds every other key**, and archiving a workspace revokes them all at once, irreversibly.

26.15 Architecture: Where the Front Door Sits



```
   |    · credential class vs endpoint  (admin? oauth?)    |
   |    · anthropic-version present and known               |
   |    · every beta name known, dated, and compatible      |
   |    · endpoint-scoped beta present on raw HTTP          |
   |    · body size under the PLATFORM's ceiling            |
   ├────────────────────────────────────────────────────────┤
   | 3  MIDDLEWARE (outward)                                |
   |    log request-id · redact · count cost · breaker      |
   ├────────────────────────────────────────────────────────┤
   | 4  TRANSPORT   retries=? timeout=? — SET THESE         |
   |    defaults: 2 retries x 600 s = 1800 s worst case     |
   |    retries draw on the SAME RPM bucket (Ch 19)         |
   ├────────────────────────────────────────────────────────┤
   | 5  MIDDLEWARE (inward)                                 |
   |    stop_reason branch (Ch 16) · refusal (Ch 25)        |
   └────────────────────────────────────────────────────────┘
                          ▼
                effect, with request-id attached
```

Three rules the picture encodes. **Preflight before transport**, because every check in step 2 is free and offline while every failure it prevents costs a round trip and a rate-limit slot. **Set the transport parameters explicitly**, because a default you did not choose is a decision you cannot review. And **resolve once, at the edge**, because a credential resolved lazily inside a request path is a credential whose source can change between two calls in the same process.

26.16 Examples

Example 1 — the smallest complete request. Three headers, three body fields, no SDK:

```
curl https://api.anthropic.com/v1/messages \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model":"claude-opus-5","max_tokens":64,
       "messages":[{"role":"user","content":"Hello"}]}'
```

Example 2 — the same call, three clients. The bodies are byte-identical; §26.21 asserts it.

```
# SDK: headers added for you
client = Anthropic()                      # reads ANTHROPIC_API_KEY
msg = client.messages.create(
    model="claude-opus-5", max_tokens=64,
    messages=[{"role": "user", "content": "Hello"}])
```

```
# CLI: flags merged over stdin, flags win
ant messages create --model claude-opus-5 --max-tokens 64 \
    --message '{role: user, content: Hello}'
```

Example 3 — a beta, three ways. Note that the SDK and CLI forms cannot make the repeated-flag mistake in the same way.

```
resp = client.beta.messages.create(
    model="claude-opus-5", max_tokens=1024,
    messages=[{"role": "user", "content": "Summarise."}],
```

```
    betas=["files-api-2025-04-14"])
```

```
ant messages create --beta files-api-2025-04-14,task-budgets-2026-03-13 \
    --model claude-opus-5 --max-tokens 1024 ...
# NOT: --beta files-api-2025-04-14 --beta task-budgets-2026-03-13
#      (only the first flag takes effect)
```

Example 4 — an Admin API call, with the other credential. Note `authorization`, not `x-api-key`, and note that this token reaches the whole organisation regardless of the workspace its profile is bound to:

```
ant auth login --profile admin --scope "org:admin"
export ANTHROPIC_OAUTH_TOKEN=$(ant auth print-credentials \
    --profile admin --access-token)
curl --fail-with-body -sS \
  "https://api.anthropic.com/v1/organizations/me" \
  -H "anthropic-version: 2023-06-01" \
  -H "authorization: Bearer $ANTHROPIC_OAUTH_TOKEN"
```

Example 5 — auditing key expiry. The audit trail from §26.4, in one call:

```
ant --format jsonl beta:organizations:api_keys list \
  --transform "{name,status,expires_at}"
```

Example 6 — what a wrong environment does. The introduction's incident, reduced:

```
# WRONG: the key is present, so it wins, and it is empty
export ANTHROPIC_API_KEY=""
export ANTHROPIC_FEDERATION_RULE_ID=fdrl_...   # never consulted
# RIGHT
unset ANTHROPIC_API_KEY
```

26.17 Code Walkthrough: The Resolver That Counts Presence

The whole of this chapter's first incident lives in one predicate. Here is `resolve` from `ch26/creds.py`, with the rung that matters annotated line by line.

```
def _present(env, name):
    """Presence, not truthiness. This one line is the whole point."""
    return name in env
```

Line 3 is the entire difference between the documented chain and the one most people implement. `name in env` is true for `""`; `env.get(name)` is not.

```
    for name, header in (("ANTHROPIC_API_KEY", "x-api-key"),
                         ("ANTHROPIC_AUTH_TOKEN", "authorization")):
        if _present(env, name):
            value = env[name]
            kind = classify(value) if header == "x-api-key" else "bearer"
            return Credential(
                kind, header, f"env:{name}", usable=bool(value),
                note="" if value else
```

```
                    "set but empty: it still shadows federation")
```

Line 3 tests presence, and **returns** — the chain stops here whether or not the value is usable. Line 6 records `usable=bool(value)` as a separate fact from "this rung won", which is the distinction the naive resolver collapses. Line 8 makes the resolver capable of explaining itself, which is what turns a four-hour incident into a ninety-second one.

```
    if _present(env, "ANTHROPIC_PROFILE"):
        name = env["ANTHROPIC_PROFILE"]
        if name not in profiles:
            raise CredentialError(
                f"ANTHROPIC_PROFILE={name!r} names a profile that does "
                f"not exist; a missing named profile is an error, not "
                f"a fall-through")
```

Line 4 raises. This is the documented behaviour — *"a missing named profile is an error, not a fall-through"* — and it is the second place the naive resolver differs, because `if env.get("ANTHROPIC_PROFILE")` followed by "load it, and if that fails try the next rung" is the natural thing to write and the wrong thing to write. A typo in a profile name should stop the process, not silently authenticate it as something else.

```
    have_all = (all(_present(env, v) and env[v]
                    for v in FEDERATION_VARS)
                and any(_present(env, v) and env[v]
                        for v in IDENTITY_VARS))
```

Note that here, at rung 4, truthiness is **correct**: the documentation says the federation path *"activates only when"* all four are set, so a blank federation variable does not arm the path. The rule is not "always use presence"; the rule is "use what the documentation says for each rung", and the two rungs differ. That asymmetry is exactly why writing it down beats remembering it.

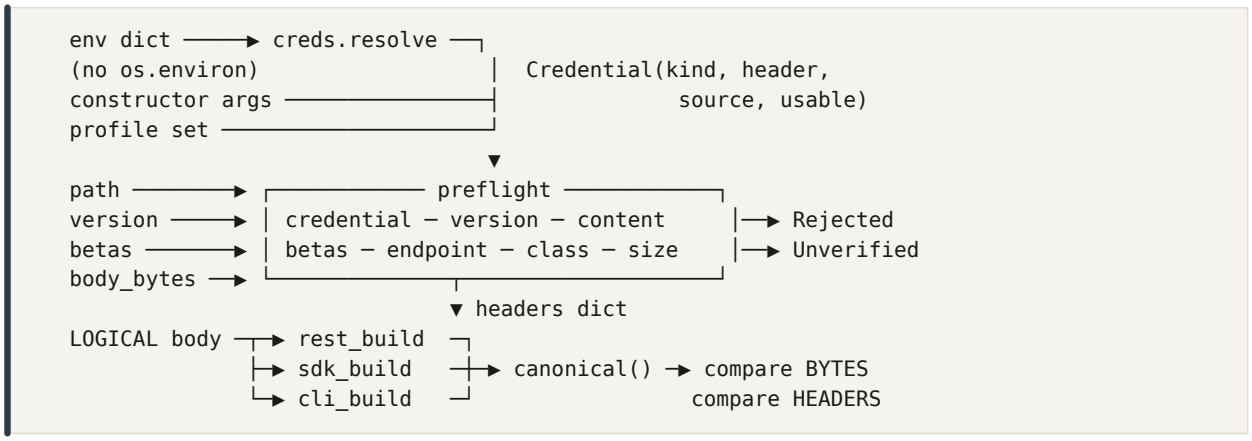
26.18 Mini Project: The Front-Door Harness

What it is. A parity harness and a preflight that together answer two questions without a key, a network or a bill: *are these three clients sending the same request?* and *which of the things I am about to send will be refused?*

Folder structure.

```
ch26/
  wire.py        copied from ch16/ — the request body and stop_reason
  registry.py    copied from ch14/ — model IDs with provenance
  creds.py       the credential precedence chain, both versions
  preflight.py   20 documented rejections + 2 UNKNOWNs, offline
  parity.py      three clients, one body; retries and timeouts
  platforms.py   five platforms, 45 cells, 4 of them UNKNOWN
  hygiene.py     leak scan over 10 artefacts + blast radius
  report.py      prints every figure in this chapter
  test_ch26.py   41 tests
```

Architecture.



Running it.

```
cd ch26 && python3 report.py          # every figure below
cd ch26 && python3 -m pytest -q       # 41 passed
```

Deployment. The preflight is the deployable half. It is pure, it needs no credential, and it takes microseconds, so it belongs in three places: as a unit test over your call sites, as an assertion inside your outbound middleware (§26.9), and as a CI check over any cURL or CLI command committed to a runbook.

26.19 Panel A — Eleven Environments, Two Resolvers

Eleven BUILT environments, each constructed to exercise a documented rule. The **documented** column is `creds.resolve`; the **naive** column is `creds.naive_resolve`, identical except for four `in`-versus-truthiness tests.

Scenario	Documented	Naive
laptop, key in the shell	`env:ANTHROPIC_API_KEY/api_key`	same
CI, federated, no key	`federation/bearer`	same
migrated to federation, key BLANKED not unset	`env:ANTHROPIC_API_KEY/empty` **EMPTY**	`federation/bearer`
blank AUTH_TOKEN over a good federation config	`env:ANTHROPIC_AUTH_TOKEN/bearer` **EMPTY**	`federation/bearer`
profile active, no key	`profile:production/bearer`	same
profile active, key ALSO set	`env:ANTHROPIC_API_KEY/api_key`	same
blank key over an active profile	`env:ANTHROPIC_API_KEY/empty` **EMPTY**	`profile:production/bearer`
profile named but never created	**RAISED**	`profile:staging/bearer`
admin key in the ordinary variable	`env:ANTHROPIC_API_KEY/admin_key`	same

Scenario	Documented	Naive
federation half-configured (no service account)	`none/none`	same
nothing set at all	`none/none`	same

The resolvers disagree on 4 of 11 scenarios. Every disagreement is a case where the naive resolver reports a *usable* credential that the client will not, in fact, use — which is precisely the shape of a bug that survives code review, because the code says the right thing and the runtime does not.

Two of the four are the empty-string rule in its two forms. The third is the same rule shadowing a profile rather than federation. The fourth is a typo in a profile name: the documented behaviour is to raise, and the naive resolver authenticates as a profile that does not exist.

KEY TAKEAWAYS

The finding in one line

Rung 2 of the chain is keyed on **presence**; rung 4 is keyed on **truthiness**. Getting one of them right and assuming the other matches is how four of eleven environments resolve to the wrong credential.

26.20 Panel B — Twenty Rejections, No API Key

`preflight.py` reproduces every documented front-door rejection offline. Each carries a provenance marker: **DOC** transcribed from the official documentation, **BUILT** constructed to exercise a documented contract.

#	Case	Prov.	Status
1	no credential at all	DOC	401 `authentication_error`
2	key present but blank	DOC	401 `authentication_error`
3	`ANTHROPIC_PROFILE` names a profile that does not exist	DOC	401 `authentication_error`
4	`anthropic-version` omitted	DOC	400 `invalid_request_error`
5	`anthropic-version` misremembered as `2024-06-01`	BUILT	400
6	`content-type` left as `text/plain`	BUILT	400
7	beta name invented	DOC	400
8	beta date off by one day	BUILT	400
9	beta date dropped entirely	BUILT	400
10	`/v1/agents` without its endpoint beta	DOC	400

#	Case	Prov.	Status
11	`/v1/sessions` without its endpoint beta	DOC	400
12	`/v1/tunnels` without its endpoint beta	DOC	400
13	`/v1/memory_stores` without its endpoint beta	DOC	400
14	`agent-memory` **and** `managed-agents` on a memory store	DOC	400
15	workspace key on the Admin API	DOC	403 `permission_error`
16	Admin API key on the service-account endpoints	DOC	403
17	Admin API key on the federation-rule endpoints	DOC	403
18	33 MB Messages request	DOC	413 `request_too_large`
19	300 MB batch	DOC	413
20	600 MB file upload	DOC	413

Reproduced: 20 of 20, offline, with no key.

And two cases the documentation covers **neither way**, which the preflight refuses to guess about:

Case	Behaviour
both `x-api-key` and `Authorization` on one request	**UNKNOWN — raises**
an Admin API key sent to `/v1/messages`	**UNKNOWN — raises**

Both are things a reasonable engineer might do while migrating. The documentation says *"one of* `x-api-key` *or* `Authorization`", which constrains the minimum and says nothing about the maximum; and it says the Admin API *requires* an admin key without saying what an admin key does elsewhere. Chapter 23's rule applies unchanged: **a fact documented in neither the supported nor the unsupported list is UNKNOWN**, and UNKNOWN must raise rather than default.

26.21 Panel C — One Request, Three Clients, 202 Bytes

The logical request is a small Messages call: Opus 5, `max_tokens` 1024, a one-block system prompt and a one-block user turn. It is built three ways.

- **Raw REST** — headers spelled out by hand, body assembled directly.
- **SDK-shaped** — `betas=[...]` becomes the header, version and content type added silently.
- **CLI** — a stdin document supplying `system`, `messages` and `model: claude-sonnet-5`, with `--model claude-opus-5` and `--max-tokens 1024` as flags.

Note that the CLI's stdin document asks for **Sonnet 5** and its flag asks for **Opus 5**. Flags take precedence, silently, and that is how the three come out identical.

Measurement	Result
body bytes identical across all three	**True**
serialised length	**202 bytes**
header sets identical	**True**

That is the good result and it deserves to be stated plainly: **an SDK is a convenience, not the protocol.** Chapter 16's claim survives contact with three clients.

Now the three places a convenience changes something anyway:

Divergence	Same body?	Same headers?	What happened
raw REST vs SDK `beta` namespace on `/v1/agents`	yes	**no**	the SDK added `anthropic-beta: managed-agents-2026-04-01`
two `--beta` flags vs one comma-separated flag	yes	**no**	*"2 --beta flags given; only the first takes effect, so 1 beta name was dropped silently"*
stdin document vs the same document plus `--model`	**no**	yes	flags take precedence over stdin, silently

Read the first row from the other direction and it is the reason a working cURL command stops working when someone "simplifies" it out of an SDK: the body is right, the header is missing, and the 400 says nothing about SDKs. Read the second row and it is a documented behaviour with no runtime signal at all — the request succeeds, one beta feature is simply absent, and whatever depended on it fails somewhere else entirely.

COMMON MISTAKE

The cURL that works and the SDK call that does not, or the reverse

Symptom. A request works from one client and 400s from another, with an identical body. **Cause.** An endpoint-scoped beta header the SDK adds for you, or a second `--beta` flag the CLI discards. **Fix.** Print the bytes. `ant --debug`, or SDK middleware that logs the outgoing headers. **Detection.** A parity test: build the request both ways and assert on the header dict, not just the body.

26.22 Panel D — The Defaults You Did Not Set

Two constructor defaults, documented on every SDK page, absent from every codebase:

Default	Value
`max_retries`	**2** — so up to **3 attempts**
`timeout`	**600 s** (10 minutes)

Worst case before the caller sees an error: 1,800 s — thirty minutes, ignoring backoff, on a client whose timeout is documented as ten minutes. Set `timeout=20.0`, as the documentation's own example does, and the same arithmetic gives 60 s.

The retries are not free in another currency either. Every attempt is a request against Chapter 19's RPM bucket. With `max_retries` retries and a per-attempt transient failure probability p, the expected attempts per logical request are $1 + p + {*p}^{*2}$, and the bucket sees all of them.

Take a service planned at **4,800 logical requests per minute** against a Build-tier limit of **5,000 RPM**:

p(transient)	factor	drawn RPM	% of bucket	over?
0.00	1.0000	4,800.0	**96.00%**	no
0.05	1.0525	5,052.0	**101.04%**	**YES**
0.30	1.3900	6,672.0	**133.44%**	**YES**

A plan sitting at 96 per cent of its bucket goes over at a **five per cent** transient failure rate, purely from a default nobody set. At thirty per cent — an overload episode, which is exactly when transient failures cluster — it is a third over. And the 429s that result are themselves retried by the same default, which is the self-amplification Chapter 19 measured directly; that chapter's finding stands, and this one explains where the extra requests came from.

Setting `max_retries=0` returns the factor to exactly 1.0 and the draw to 4,800 RPM. That is not a recommendation to disable retries — Chapter 29 will argue for a considered policy — it is a demonstration that the number is yours to choose and currently is not.

KEY TAKEAWAYS

The finding in one line

`max_retries=2` and `timeout=600` are decisions your application made by not making them: **30 minutes of worst-case latency and a 5.25 per cent tax on your rate limit at a 5 per cent failure rate.**

26.23 Panel E — Six Leaks, Two Scanners, Eighteen Operations

Ten BUILT artefacts, each a place credentials really end up. Six contain a genuine credential. The **naive** scanner matches the `sk-ant-api` prefix — the one a working developer sees daily. The **full** scanner matches all three documented prefixes.

Artefact	Where it ends up	Leak?	Class	Naive	Full
`bundle.js`	every browser that loads the page	yes	`api_key`	found	found
`Dockerfile`	an image layer and every mirror	yes	`api_key`	found	found
`ci-debug.log`	the CI log retention window	**no**	–	–	–
`analysis.ipynb`	the repository, in an output cell	yes	`admin_key`	**missed**	found

Artefact	Where it ends up	Leak?	Class	Naive	Full
`terraform.tfstate`	the state bucket	yes	`bearer`	**missed**	found
`error.json`	an issue tracker	**no**	–	–	–
`config.yaml`	the repository	**no**	–	–	–
`incident-thread.txt`	a chat workspace, forever	yes	`api_key`	found	found
`README.md`	the repository	**no**	–	–	–
`.env.local`	the client bundle, via the bundler	yes	`api_key`	found	found

Real leaks 6; naive found 4; full found 6. Neither scanner produced a false positive.

Three of the true negatives are worth dwelling on. `ci-debug.log` is a transcript of `ant --debug`, which **redacts the key** — a point in the CLI's favour and a reason to prefer it over hand-rolled request logging. `config.yaml` contains `api_key: ${ANTHROPIC_API_KEY}`, a *reference* rather than a secret. `error.json` contains a `request_id` that looks alarming and is not a credential at all.

Now the reason the two misses matter. Eighteen documented operations against four credential classes:

Class	Reaches	UNKNOWN	of
`ws_key` — a workspace API key	6	0	18
`ws_oat` — a `workspace:developer` federation token	6	0	18
`admin_key` — an Admin API key	**9**	6	18
`admin_oat` — an `org:admin` OAuth token	**11**	7	18

The workspace credentials reach six operations — Messages, token counting, batches, files, skills, models — **and only within one workspace**. The Admin API key reaches nine administrative ones, including listing every key in the organisation, **deactivating** any of them, changing a member's role, removing a member, and archiving a workspace, which *"immediately revokes all API keys in that workspace"* and *"cannot be undone"*. The `org:admin` OAuth token adds the service-account and federation-rule endpoints, which is the ability to mint new identities.

The naive scanner misses exactly the two classes with the largest blast radius. That is not bad luck: it is the predictable consequence of writing a pattern from experience rather than from the documented list, and the documented list has three entries.

The UNKNOWN columns are honest gaps. The scope table documents `workspace:developer` reaching Messages; it does not say whether an `org:admin` token or an Admin API key can call `/v1/messages`, and this book will not guess. `hygiene.can()` raises on those cells.

KEY TAKEAWAYS

The finding in one line

The credential prefix you have never seen is the one with the master key. A leak scanner built from three documented prefixes finds 6 of 6; one built from the prefix you use daily finds 4 of 6 and misses both privileged classes.

26.24 Panel F — Forty-Five Cells, Four of Them Unknown

Nine fields across five platforms. **Verified 41 of 45 cells on 1 August 2026; 4 are UNKNOWN and raise** rather than defaulting to the Claude API's value.

The four gaps are Microsoft Foundry's model-ID format and request-size ceiling, and the Files API on Foundry and on Claude Platform on AWS. Each could plausibly be inferred — Foundry is Anthropic-operated, so "probably the Claude API format and 32 MB" is a reasonable guess — and each is exactly the guess Chapter 14 forbade.

Portability of a request by size:

Payload	Accepted by	Refused by	Unverifiable on
18 MB	4 platforms	–	Foundry
25 MB	3 platforms	**Bedrock**	Foundry
31 MB	2 platforms	**Bedrock, Google Cloud**	Foundry

A 25 MB request — a modest batch of PDFs by Chapter 17's arithmetic — is legal on the Claude API and Claude Platform on AWS, a `413 request_too_large` on Bedrock, legal on Google Cloud, and unverifiable on Foundry. "It works in our environment" is a statement about one column of this table.

26.25 Evaluation and Deployment

The harness has no model in it, so there is nothing to evaluate statistically; what there is to evaluate is coverage, and the honest measures are these:

Measure	Value
Documented rejections reproduced offline	20 of 20
UNKNOWN cases that raise rather than guess	2 of 2
Platform cells verified	41 of 45
Credential classes recognised	3 of 3 documented prefixes
Tests	41, passing, no key, no network

Deployment has three placements, in increasing order of value. **In tests**, assert that your call sites survive `preflight`. **In middleware**, run it on every outbound request in staging and log rather than raise, so that

a beta name that drifts out of date announces itself before a customer does. **In CI**, run it over every cURL and `ant` command committed to a runbook, because runbooks are where stale beta dates go to live.

26.26 Hands-on Labs

Lab 26.1 — Resolve a credential from an environment. Write a function taking an env dict and returning which rung wins. Include a case with `ANTHROPIC_API_KEY=""` alongside a complete federation configuration. *Success criterion:* your function returns the API-key rung, marked unusable, and your test asserts it — not the federation rung.

Lab 26.2 — Reproduce five rejections offline. Pick five rows from Panel B and implement the checks. *Success criterion:* five tests pass with no network call and no key, and each asserts the HTTP status, not merely that an exception was raised.

Lab 26.3 — Prove parity between two clients. Build one logical request with an SDK-shaped helper and with raw REST. Serialise both canonically. *Success criterion:* the bodies are byte-identical and your test additionally compares the header dicts and reports any difference by name.

Lab 26.4 — Price your own retry defaults. Take your service's real RPM and rate-limit tier. Compute the drawn RPM at p = 0, 0.05 and 0.30 under `max_retries=2`. *Success criterion:* you can state the p at which your plan crosses its bucket, to two decimal places.

Lab 26.5 — Scan for all three prefixes. Write a scanner over a directory that matches `sk-ant-api`, `sk-ant-admin` and `sk-ant-oat01`, and report the credential class of each hit. *Success criterion:* it finds a planted Admin API key and an OAuth token, and does **not** flag `${ANTHROPIC_API_KEY}` or a redacted `<REDACTED>` transcript.

26.27 Lab Solutions

26.1. The whole solution is `if name in env:` rather than `if env.get(name):`, plus recording `usable=bool(value)` as a separate field so that "which rung won" and "can it authenticate" stay distinct. Return an object, not a string; the incident in the introduction was solved by an object that could explain itself. See `creds.resolve`.

26.2. Order the checks credential → version → content type → betas → credential-class-versus-endpoint → size, and return the surviving header dict on success so the function is useful rather than merely defensive. Assert `exc.status`, because a test that only asserts `pytest.raises(Rejected)` passes when you reject the right request for the wrong reason. See `preflight.preflight`.

26.3. Canonicalise with `json.dumps(body, sort_keys=True, separators=(",", ":")).encode()`. Sorting keys is what makes "the same request" testable across clients that build the dict in different orders — `test_canonical_is_stable_under_key_order` asserts exactly that. Then compare headers separately: in this chapter's run the bodies matched at 202 bytes and the headers diverged in two of three scenarios. See `parity.canonical` and `parity.parity_run`.

26.4. factor = $*p^*$ for k = 0..`max_retries`; drawn = logical × factor. For 4,800 RPM against 5,000: crossing happens between p = 0.00 and p = 0.05, and solving $4800(1 + p + {*p}^{*2}) = 5000$ gives $p \approx$ **0.0401**. Anything above a four per cent transient failure rate puts that plan over. See

`parity.retry_amplification`.

26.5. Match on all three prefixes and classify the hit; do not attempt to validate the remainder of the string, because the prefixes are the documented part and the suffix format is not. Treat a `${VAR}` reference and a `<REDACTED>` placeholder as negatives by checking that the character following the prefix is not `}` or `<` — or, more simply and more robustly, by requiring the documented separator that follows each prefix. See `hygiene.scan`.

26.28 Best Practices

BEST PRACTICE

Unset, never blank

When you migrate off a credential, remove the environment variable. A blank variable occupies its rung and shadows everything below it, and Panel A measures the cost at four of eleven environments.

BEST PRACTICE

Resolve once, at the edge, and log the source

Resolve the credential when the client is constructed, log which rung won and which workspace it is bound to, and never resolve lazily inside a request path. `ant auth status` is the model: it reports the source, the profile and the workspace, and it exists because people need to know.

BEST PRACTICE

Set `max_retries` and `timeout` explicitly, even to their defaults

A default you have written down is a decision you can review; a default you inherited is one you cannot. Writing `max_retries=2, timeout=600` changes nothing about behaviour and everything about the next incident review.

BEST PRACTICE

Print the bytes from one place

Put outbound logging in middleware (§26.9), not at call sites. One place to read, one place to redact, and one place that will show you the beta header the SDK added on your behalf.

BEST PRACTICE

Prefer federation for anything that is not a laptop

The documented split is unambiguous: API keys for *"local development, prototyping, scripts, and single-tenant servers where you control secret storage"*, federation for *"production workloads on cloud platforms, CI/CD pipelines, and Kubernetes"*. A credential a workload mints for itself cannot be committed to a repository.

BEST PRACTICE

Give every key an expiration, or a rotation job — and audit `expires_at`

Expiration cannot be changed after creation, and keys shorter than seven days expire **with no warning email**. Enumerate `expires_at` from the Admin API on a schedule and alert on both imminent expiry and `null`.

BEST PRACTICE

One workspace per environment, and know what that splits

Workspace separation gives you per-environment keys, rate limits and spend limits, and it splits your Files, Batches, Skills **and your prompt cache** on the Claude API. That last one is a cost decision; make it deliberately.

BEST PRACTICE

Use a dedicated profile for admin access

The documented pattern is `ant auth login --profile admin --scope "org:admin"`, precisely so *"a dedicated profile keeps your routine commands from running with elevated access."* An `org:admin` token reaches the whole organisation regardless of the workspace its profile is bound to.

BEST PRACTICE

Catch typed SDK exceptions, most specific first

Never string-match an error message. The message is prose and may change; the type is contract, and even the type list is documented to grow.

BEST PRACTICE

Send `anthropic-version: 2023-06-01` and never override it

The SDKs do it for you and warn that overriding *"may result in incorrect types and other unexpected or undefined behavior."* There is no scenario in this book where changing it helps.

BEST PRACTICE

Treat every enum as open

Versioning explicitly reserves the right to add new values to enum-like outputs. `stop_reason`, streaming event types and error `type` values are all in that category. Raise on the unknown value; do not fall through.

BEST PRACTICE

Pin beta names with their dates in one constant

A beta name is `feature-YYYY-MM-DD` and the date is part of it. Put the list in one module with a comment recording where each was verified and when, so that a stale name is a one-line diff rather than an archaeology exercise.

BEST PRACTICE

Never ship a credential to a client

The TypeScript SDK runs in the browser, which makes this easy to get wrong. A key in a bundle is a key in every browser that loads the page, and no expiration is short enough to make that acceptable.

BEST PRACTICE

Record the request ID on every call, success or failure

`request-id` is on every response and inside every error body. On Claude Platform on AWS, record both it and `x-amzn-requestid`. Support tickets and CloudTrail lookups need different ones.

26.29 Common Mistakes

COMMON MISTAKE

Blanking a credential variable instead of unsetting it

Symptom. `401 authentication_error` on every request after a migration to federation, with no exchange attempts in the Console's authentication history. **Cause.** `ANTHROPIC_API_KEY=""` is present, so it wins rung 2 of the precedence chain and the federation path is never armed. **Fix.** Unset the variable. Do not blank it. **Detection.** Log which rung won at client construction; or run `ant auth status`, which prints the credential source that was selected.

COMMON MISTAKE

Assuming a profile name that does not exist falls through

Symptom. A typo in `ANTHROPIC_PROFILE` produces requests attributed to the wrong workspace, or an unexplained 401. **Cause.** *"A missing named profile is an error, not a fall-through"* — but a hand-rolled resolver usually treats it as one. **Fix.** Raise. Panel A shows the naive resolver silently accepting `profile:staging` when only `production` exists. **Detection.** A test that asserts the resolver raises for an unknown profile name.

COMMON MISTAKE

Repeating the `--beta` flag on the CLI

Symptom. One beta feature behaves as though it were not enabled; the request otherwise succeeds. **Cause.** *"Avoid repeating the flag: currently only the first flag's value takes effect."* **Fix.** One `--beta` flag, comma-separated. **Detection.** `ant --debug` prints the `Anthropic-Beta` header actually sent.

COMMON MISTAKE

Copying a cURL command out of an SDK integration

Symptom. A 400 on a beta endpoint from cURL, with the identical body that the SDK sends successfully. **Cause.** The SDKs' `beta` namespaces add the endpoint-scoped header (`managed-agents-2026-04-01` and friends) automatically. Raw HTTP callers must add it themselves. **Fix.** Add the header. Panel B reproduces four such rejections. **Detection.** A preflight that knows which paths require which beta.

COMMON MISTAKE

Sending both memory-store beta headers

Symptom. A 400 on `/v1/memory_stores` after adding agent memory to a Managed Agents integration. **Cause.** On memory-store endpoints `agent-memory-2026-07-22` **replaces** `managed-agents-2026-04-01`; sending both is an error. **Fix.** Send one, per endpoint. The SDKs do this for you. **Detection.** The mutually-exclusive check in `preflight.py`.

COMMON MISTAKE

Inheriting `max_retries` and `timeout`

Symptom. A "ten-minute" client that takes thirty minutes to fail; a rate-limit tier exceeded during an incident with no change in application traffic. **Cause.** `max_retries=2` and `timeout=600` are constructor defaults. **Fix.** Set both explicitly to values you can defend. Panel D prices the difference. **Detection.** Alert on p100 request duration exceeding your configured timeout — if it does, you are seeing retries.

COMMON MISTAKE

Assuming a `file_id` is portable across workspaces

Symptom. A `404 not_found_error` in production on a request that works in development, referencing a valid file. **Cause.** Files, Batches and Skills are scoped to the workspace of the key that created them. **Fix.** Upload per workspace, or key your file cache by workspace. **Detection.** Include the workspace in the cache key and in the log line, so the 404 is self-explaining.

COMMON MISTAKE

Writing a leak scanner from memory

Symptom. A clean secret-scanning report and a leaked Admin API key. **Cause.** The scanner matched `sk-ant-api` only. There are three documented prefixes. **Fix.** Match `sk-ant-api`, `sk-ant-admin` and `sk-ant-oat01`. Panel E measures 4 of 6 versus 6 of 6. **Detection.** Plant one credential of each class in a fixture and assert your scanner finds all three.

COMMON MISTAKE

Using an Admin API key for the federation endpoints

Symptom. `403 permission_error` when creating a service account or federation rule with a working Admin API key. **Cause.** Service-account, federation-issuer and federation-rule endpoints accept **only** an `org:admin` OAuth token. **Fix.** `ant auth login --profile admin --scope "org:admin"` and use the bearer token. **Detection.** Panel B, rows 16 and 17.

COMMON MISTAKE

Assuming request-size limits are the same everywhere

Symptom. A 413 on Bedrock for a payload that works on the Claude API. **Cause.** Bedrock limits requests to 20 MB and Google Cloud to 30 MB; the Claude API and Claude Platform on AWS allow 32 MB. **Fix.** Size to the smallest platform you target. Panel F shows a 25 MB request accepted by three of five. **Detection.** A per-platform ceiling in the preflight, not a single constant.

COMMON MISTAKE

Treating a 401 on Claude Platform on AWS as a key problem

Symptom. A 401 with no Anthropic credential involved. **Cause.** There, a 401 *"can also indicate a problem with your AWS credentials or SigV4 signature."* **Fix.** Check the AWS side. If every request fails with *"Outbound web identity federation is disabled"*, run `aws iam enable-outbound-web-identity-federation` once per account. **Detection.** Log `x-amzn-requestid` alongside `request-id` and look the former up in CloudTrail.

COMMON MISTAKE

Building alerting on error type strings

Symptom. An alert stops firing, or a new error type falls through a `match` with no arm. **Cause.** *"It is possible that the* `type` *values will grow over time."* **Fix.** Branch on status code and typed exception class; treat an unrecognised type as an error, not as a default case. **Detection.** A test that feeds an invented error type through your handler and asserts it does not silently succeed.

26.30 Performance Considerations

The front door contributes almost nothing to latency and a great deal to tail latency, and the difference is entirely in the transport defaults.

The preflight is free. Every check in Panel B is a dictionary lookup or a string comparison; the whole corpus of twenty runs in microseconds. A round trip to be told the same thing costs tens to hundreds of milliseconds, a rate-limit slot, and in the 413 case does not even reach Anthropic's servers.

Retries are the tail. Panel D's thirty-minute worst case is not a latency figure anyone would accept if it were written down, and the point is that it is not written down anywhere in a typical codebase. If your p100 exceeds your configured timeout you are looking at retries, and that is the cheapest possible detector.

Federation adds an exchange, once. The SDK caches the access token and refreshes before expiry, so with the default 3,600-second lifetime the cost is one extra round trip per hour per process. Shortening `token_lifetime_seconds` to its 60-second minimum turns that into one per minute; there is a security argument for it and a latency price attached.

Connection reuse matters more than any of this. The SDKs set a TCP keep-alive socket option specifically to reduce idle-connection timeouts, and replacing the HTTP client without preserving the SDK's defaults gives that up. In Python, `DefaultHttpxClient` exists for this reason.

Pagination is a hidden multiplier. Auto-pagination is convenient and forward-only; a `limit` of 20 over a list of 2,000 is 100 round trips inside what looks like one `for` loop.

26.31 Security Considerations

The blast radius is the design input. Panel E's eighteen-operation matrix is the security model in one table: a workspace key reaches six operations in one workspace, an Admin API key reaches nine organisation-wide ones including deactivating every other key, and an `org:admin` token can mint new identities. Choose the weakest credential that does the job, and make the strong ones hard to obtain — the documented pattern of a dedicated `admin` profile exists exactly for this.

Nothing goes in a client. The TypeScript SDK supports the browser, which means the mistake is one import away. A key in a bundle is a key for everyone. If a browser needs Claude, it needs your server in front of it.

Expiration limits a leak; it does not prevent one. The documentation says so directly: *"Expiration limits the lifetime of a leaked credential, but it is not a substitute for secret hygiene."* Store in a secrets manager, rotate, and revoke anything you suspect.

Federation moves the secret; it does not delete it. A static cloud credential upstream that can mint IdP tokens undermines the chain. Pair federation with IP allowlists, MFA and audit logging on the provider side.

CEL conditions are a security boundary. A federation rule's `condition` that evaluates true for more inputs than intended grants broader access than intended, and the documented advice is to prefer the static matchers when they express your constraint. The rule that a `match` block containing only `audience` is **rejected** exists because such a rule would accept every token from the issuer.

JWKS rotation has a one-minute window. Publish new signing keys at least fifteen minutes before signing with them, and keep superseded keys until tokens they signed expire.

Redact at the boundary, and prefer tools that already do. `ant --debug` redacts the key. Hand-rolled request logging usually does not, and Panel E's `ci-debug.log` is a true negative only because the CLI wrote it.

Archiving is irreversible and immediate. Archiving a workspace revokes every key in it and cannot be undone. That is a legitimate incident-response tool and a spectacular way to cause an outage.

26.32 Real Production Examples

A fintech's four-hour 401. The incident in the introduction. Root cause: `ANTHROPIC_API_KEY=""` left in a deployment manifest. The fix was one line; the lesson was that the client could not say which credential it had chosen. They added a startup log line naming the rung and the workspace, and the same class of misconfiguration has since been caught twice at deploy time.

A media company's 413 on Bedrock only. A document-processing pipeline that had run happily on the Claude API for months was extended to Bedrock for a customer with an AWS commitment. Payloads averaging 24 MB — well inside 32 MB — began returning `413 request_too_large` on Bedrock alone. The preflight now carries a per-platform ceiling and the pipeline chunks to the smallest target, which is 20 MB.

A SaaS vendor's rate-limit incident with no traffic change. A partial upstream outage produced a modest rate of connection errors. Application RPM did not move; observed RPM rose by roughly a third and the organisation started taking 429s. The cause was `max_retries=2` on every client in the fleet, compounding a transient failure rate into bucket pressure — Panel D's third row. They set retries explicitly per service and added a p100-versus-timeout alert.

A consultancy's clean scan and leaked master key. A secret-scanning rule matching `sk-ant-api` reported zero findings for eight months. An Admin API key had been committed inside a notebook output cell the whole time. The rule now matches all three documented prefixes, and the repository history was rewritten and the key revoked.

26.33 Certification Notes

Concept	Domain	Weight	What is examinable
The three required headers	D4 Prompt Engineering & Structured Output	20%	`x-api-key` **or** `Authorization`, `anthropic-version`, `content-type`
Credential precedence	D5 Context Management & Reliability	15%	constructor > env key/token > profile > federation; a blank variable still counts
API key vs federation	D5	15%	when each is documented as appropriate; both grant the same access
Workspace scoping	D5	15%	Files, Batches and Skills are workspace-scoped; keys carry no rate limits
Admin API credentials	D5	15%	`sk-ant-admin` key or `org:admin` OAuth; federation endpoints take only the token
`anthropic-version` semantics	D4	20%	inputs and outputs preserved; new enum values may appear
`anthropic-beta` mechanics	D4	20%	comma-separated, dated names, endpoint-scoped headers, the mutually exclusive pair
The seven SDKs and the CLI	D2 Tool Design & MCP Integration	18%	what each adds; the `beta` namespace adds headers for you

Concept	Domain	Weight	What is examinable
SDK middleware	D1 Agentic Architecture & Orchestration	27%	ordering; client before per-request; HTTP client replaces, runs last
Retry and timeout defaults	D5	15%	2 retries, 10-minute timeout, `retry-after` honoured
Platform differences	D1	27%	three ID formats, three request ceilings, two cache scopes
HTTP error shape	D5	15%	`type`/`message` plus `request_id`; catch typed classes, not messages

CERTIFICATION NOTE

Exam tips

The header set is the single highest-yield fact: `x-api-key` **OR** `Authorization`**, plus** `anthropic-version`**, plus** `content-type`**.** If a question lists all four as required, it is wrong — the first two are alternatives.

`anthropic-version: 2023-06-01` is the value. Learn what it promises: existing inputs and outputs preserved, **new enum values permitted**.

Beta questions almost always turn on one of three things: the **date is part of the name**, the **endpoint-scoped headers** the SDKs add for you, or the `agent-memory` / `managed-agents` **conflict** on memory stores.

If a question says an API key returned 403 on an organisation endpoint, the answer is that the **Admin API needs an Admin API key or an** `org:admin` **OAuth token**. If it says an *Admin API key* returned 403, look for **service accounts or federation** — those take only the OAuth token.

Remember that **API keys carry no rate limits of their own** (Chapter 19) but **do** scope Files, Batches and Skills.

Retry and timeout defaults — **2 and 10 minutes** — are examinable numbers.

And if a question describes a migration to federation that 401s, the answer is a **credential variable that was blanked rather than unset**.

26.34 Twenty Practice Questions

1. Name every header required on a Claude API request, and say which two are alternatives.
2. What are the two documented authentication methods, and does either grant more access than the other?
3. List the four rungs of the credential precedence chain in order.
4. `ANTHROPIC_API_KEY=""` is exported alongside a complete federation configuration. Which credential is used?
5. `ANTHROPIC_PROFILE` names a profile that does not exist. What happens?
6. Which environment variables must all be set for the federation path to activate?
7. When a profile is loaded, do environment variables override the fields it sets?

8. What expiration options exist when creating an API key, and can expiration be changed later?
9. Which keys expire without a warning email?
10. Which three resource types are scoped to a workspace?
11. On which platforms is the prompt cache scoped per organisation rather than per workspace?
12. What does `anthropic-version` preserve, and what does it explicitly permit to change?
13. How do you send two beta features with the CLI, and what happens if you repeat `--beta`?
14. Which beta header is required on `/v1/memory_stores`, and which one must **not** accompany it?
15. Name the seven official SDKs.
16. In what order does middleware run, and where does a custom HTTP client sit relative to it?
17. What are the default `max_retries` and `timeout`, and what worst-case wall clock do they imply?
18. Which credential do the service-account and federation-rule endpoints accept?
19. What are the request-size ceilings on the Claude API, Bedrock and Google Cloud?
20. Which two request IDs does Claude Platform on AWS return, and what is each for?

26.35 Answers and Explanations

1. `x-api-key` **or** `Authorization`**, plus** `anthropic-version` **and** `content-type`**.** "All four are required" is the standard distractor; the credential headers are alternatives. "Only `x-api-key`" is wrong since federation exists.
2. **A static API key, or Workload Identity Federation.** *"Both methods grant the same access to Claude API endpoints."* Any answer claiming federation grants more or less is wrong; the difference is secret management.
3. **Constructor argument, then** `ANTHROPIC_API_KEY`/`ANTHROPIC_AUTH_TOKEN`**, then** `ANTHROPIC_PROFILE`**, then federation variables.** Putting the profile above the environment variables is the common inversion and is wrong.
4. **The empty API key.** A variable set to the empty string still occupies its slot. "Falls through to federation" is the intuitive answer and the documented mistake.
5. **It is an error, not a fall-through.** Answers describing a fall back to `default` or to federation are wrong.
6. `ANTHROPIC_FEDERATION_RULE_ID`, `ANTHROPIC_ORGANIZATION_ID`, `ANTHROPIC_SERVICE_ACCOUNT_ID`, **and one of** `ANTHROPIC_IDENTITY_TOKEN_FILE` / `ANTHROPIC_IDENTITY_TOKEN`**.** `ANTHROPIC_WORKSPACE_ID` is read alongside but does **not** gate activation — that is the distractor.
7. **No.** Environment variables fill fields the profile omits but never override fields it sets. "Environment always wins" is the wrong instinct here.
8. **Presets of 3 hours, 1 day, 7 days or 30 days, a custom duration, or Never; and no, expiration is set at creation and cannot be changed.** "Extend it in the Console" is wrong.
9. **Keys with a lifetime under 7 days.** Warning emails go out 7 days ahead for lifetimes of at least 14 days and 1 day ahead for at least 7.
10. **Files, Message Batches and Skills.** Naming prompt caches here is close but they are described separately; naming rate limits is wrong, since those are organisational.

11. **Amazon Bedrock and Google Cloud.** The Claude API, Claude Platform on AWS and Microsoft Foundry scope caches per workspace.
12. **Preserves existing input and output parameters; permits new optional inputs, new output values, changed error conditions, and new variants of enum-like output values.** The last is the one that matters.
13. **One `--beta` flag with comma-separated names.** Repeating the flag means only the first takes effect — silently.
14. `agent-memory-2026-07-22` **is required;** `managed-agents-2026-04-01` **must not accompany it** on memory-store endpoints. Sending both returns 400.
15. **Python, TypeScript, C#, Go, Java, PHP, Ruby.** Adding Rust or Swift is the distractor; the Apple package is a library, not a general-purpose Messages client.
16. **In the order given — first middleware's "before" runs first and its "after" runs last; client middleware before per-request middleware; the custom HTTP client receives requests after all middleware has run.** "The HTTP client wraps the middleware" is the inversion.
17. `max_retries=2` **and** `timeout=600 s`**, so up to three attempts and a 1,800-second worst case.** Answering "ten minutes" ignores the retries.
18. **An** `org:admin` **OAuth bearer token only.** Admin API keys are refused there — the counter-intuitive answer, and the correct one.
19. **32 MB, 20 MB and 30 MB respectively.** Claude Platform on AWS matches the direct API at 32 MB.
20. `x-amzn-requestid` **— primary, indexed in CloudTrail — and** `request-id` **— secondary, for Anthropic support tickets.**

26.36 Ten Practical Exercises

1. Log which credential rung wins at client construction in one of your services, then deliberately blank a variable and confirm the log tells you.
2. Run `ant auth status` and identify the credential source, the active profile and the bound workspace.
3. Run `ant --debug` against any beta resource and read the `Anthropic-Beta` header the CLI added for you.
4. Enumerate every API key in your organisation with the Admin API and list those whose `expires_at` is `null`.
5. Upload a file with a development key and attempt to reference its `file_id` with a production key. Record the status code.
6. Compute the *p* at which your service's RPM plan crosses its rate-limit bucket under `max_retries=2`.
7. Set `max_retries` and `timeout` explicitly in one service and confirm no behaviour changed.
8. Write a middleware that logs the request ID and the outgoing header names for every call.
9. Plant one credential of each documented prefix in a fixture directory and run your secret scanner over it.
10. Take the largest request your system sends and check it against the ceiling of every platform you might one day target.

26.37 Five Challenge Labs

1. **The parity CI check.** Build a test that constructs your ten most important requests through both your SDK wrapper and a raw REST builder, canonicalises both, and fails CI on any byte or header difference not explicitly allow-listed.
2. **The credential explainer.** Write a `whoami` command for your service that prints the credential rung, class, bound workspace and, for federation, the rule and service account — without printing the secret.
3. **The beta drift monitor.** Put every beta name your codebase sends into one module with a verification date, and add a CI job that fails when any entry is more than ninety days old and unreviewed.
4. **The blast-radius linter.** Given a credential class and an endpoint, decide statically whether the call can possibly succeed, and fail the build on any call site that requires more privilege than the service is configured with.
5. **The retry budget.** Implement a per-request retry budget shared across your fleet, so that the total retry rate cannot exceed a fixed fraction of your RPM bucket regardless of how many clients are failing at once.

26.38 Ten Interview Questions

1. A team migrated to Workload Identity Federation and every request 401s, with no exchange attempts logged. Where do you look first?
2. Two engineers send the same body, one from cURL and one from the Python SDK. One gets a 400. What are the likely causes?
3. When would you choose an API key over federation in production, and how would you defend it?
4. What does `anthropic-version` guarantee, and how should that shape the way you write a response handler?
5. Your organisation crossed its rate limit during an upstream incident with no change in application traffic. Explain how.
6. Why can an Admin API key create a workspace but not a federation rule?
7. A `file_id` that works in staging returns 404 in production. Walk me through it.
8. Design a secret-scanning rule for this platform. What does it match, and how do you know it is complete?
9. Where in a service would you put outbound request logging, and why there?
10. You are asked to support Bedrock in addition to the Claude API. What changes at the front door?

26.39 Summary

Chapter 16 taught the request. This chapter taught everything around it, and the material turns out to be less about protocol than about attribution: at every point between your code and Anthropic's servers, something has decided on your behalf, and the discipline is knowing what and being able to prove it.

Authentication is two credentials with equal power and opposite operational properties. A static `sk-ant-api` key is simple, portable and permanent until you revoke it; a federation bearer token is minted from an identity the workload already holds, expires within a day, and cannot be committed to a repository. The choice between them is documented and unsurprising. What is surprising — and what cost this chapter's opening incident four hours — is the chain that decides between them when both are configured. It runs constructor, environment variable, profile, federation, and it tests **presence** at the second rung and **truthiness** at the fourth. Eleven environments through both a documented and a naive resolver disagree on **four**, and every disagreement reports a usable credential the client will not actually use.

Versioning and betas are contracts with fine print. `anthropic-version: 2023-06-01` preserves your existing inputs and outputs and explicitly reserves the right to add new values to enum-like outputs — which is the documented reason Chapter 16's `stop_reason` handler and Chapter 17's SSE parser both raise on the unfamiliar rather than falling through. `anthropic-beta` is a comma-separated list of dated names, some of them required on particular endpoints regardless of what features you are using, and one pair of which is mutually exclusive. Twenty documented rejections across credential, version, content type, beta, credential-class and size were reproduced offline with no key at all, and two more cases the documentation covers neither way were made to raise rather than guessed at.

The clients are, on the evidence, faithful. One logical request built three ways — raw REST, an SDK-shaped call, and the CLI's flags-over-stdin merge — serialised to **identical 202-byte bodies**. An SDK is a convenience, not the protocol, and that claim now has a test behind it. But three conveniences change something without saying so: the `beta` namespace adds an endpoint header, a repeated `--beta` flag silently drops everything after the first, and CLI flags overrule a stdin document. Two more live in the constructor: `max_retries=2` and `timeout=600` mean a thirty-minute worst case and, at a five per cent transient failure rate, turn a plan sitting at 96.00 per cent of its RPM bucket into one drawing 101.04 per cent of it. Chapter 23 found the first member of this family in schema transformation; this chapter found five more, and the response to all six is the same — put the bytes somewhere you can read them, which middleware and `ant --debug` both do for free.

Finally, credentials leak, and the scanner most people write is blind to the ones that matter. Six real leaks across ten artefacts: the naive `sk-ant-api` pattern finds four with no false positives, and the three-prefix pattern finds six. The two it misses are an Admin API key and an OAuth token, which across eighteen documented operations reach nine and eleven respectively against a workspace key's six — including deactivating every other key in the organisation and minting new identities. Workspaces scope Files, Batches, Skills and, on Anthropic-operated platforms, the prompt cache; keys carry no rate limits of their own, because those live on the organisation. And across five platforms, forty-one of forty-five cells were verified and four remain UNKNOWN and raise, because a 25 MB request that is legal on three platforms and a 413 on a fourth is exactly the kind of fact that does not survive being assumed.

KEY TAKEAWAYS

1. **Every request needs a credential, a version and a content type.** `x-api-key` and `Authorization` are alternatives, not both; `anthropic-version` and `content-type` are always required.
2. **Both credentials grant the same access.** API keys for laptops, scripts and single-tenant servers; federation for production, CI and Kubernetes. The difference is secret management, not power.
3. **The precedence chain is keyed on presence, not truthiness.** A variable set to `""` occupies its rung and shadows everything below it; four of eleven environments resolve differently under the naive rule.
4. **A named profile that does not exist is an error, not a fall-through.** Raise; do not silently authenticate as something else.
5. **Versioning preserves fields and permits new enum values.** Treat `stop_reason`, streaming event types and error `type` as open sets, and raise on the unfamiliar.
6. **The date is part of a beta name**, some betas are required per endpoint, and `agent-memory` and `managed-agents` conflict on memory stores.
7. **An SDK is a convenience, not the protocol** — three clients, one logical request, 202 identical bytes.
8. **A convenience that changes the bytes without telling you is the failure mode of this whole layer.** Schema transformation, auto-added beta headers, a discarded `--beta` flag, flags overruling stdin, and two constructor defaults: six instances so far.
9. **The defaults you did not set have a price**: 2 retries × a 10-minute timeout is 30 minutes of worst-case latency, and 5.25 per cent more RPM at a 5 per cent failure rate.
10. **Blast radius is the security model.** Three documented prefixes, three very different reaches — 6, 9 and 11 operations of 18 — and a scanner built from the prefix you see daily misses both privileged classes.

26.40 Glossary

Term	Definition
API key	A static secret beginning `sk-ant-api`, created in the Console, sent in `x-api-key`, scoped to one workspace.
Admin API key	A static secret beginning `sk-ant-admin`, provisioned only by organisation admins, granting organisation-level management.
`anthropic-beta`	A comma-separated request header opting into pre-GA features by dated name.
`anthropic-version`	A required header pinning request and response shape; currently `2023-06-01`.
Beta namespace	The `beta` property on each SDK client, which adds endpoint-scoped beta headers automatically.
Blast radius	The set of operations a leaked credential can perform; the security-relevant property of a credential class.
Credential precedence chain	The documented order in which a client selects a credential: constructor, environment variable, profile, federation.
Endpoint-scoped beta	A beta header required on every request to a particular endpoint, independent of feature use.

Term	Definition
Federation issuer	A registered OIDC provider (`fdis_...`) whose tokens may assert workload identity.
Federation rule	A rule (`fdrl_...`) mapping issuer tokens to a service account and an OAuth scope.
GJSON path	The expression language used by the CLI's `--transform` flag to reshape responses.
Middleware	An SDK hook running before a request is sent and after the response returns; the documented extension point.
`org:admin`	The OAuth scope granting organisation-wide administrative access, including federation endpoints.
Profile	A named configuration file read by the SDKs and the CLI, holding federation parameters, workspace and base URL.
Request ID	The `request-id` response header, also present as `request_id` in error bodies; the identifier support asks for.
Service account	A non-human identity (`svac_...`) that federation tokens act as.
Token exchange	`POST /v1/oauth/token`, trading an IdP JWT for a short-lived `sk-ant-oat01` access token.
Workload Identity Federation	Authentication by exchanging an identity-provider JWT for a short-lived Claude API token, eliminating static secrets.
Workspace	A container within an organisation scoping API keys, Files, Batches, Skills and — on Anthropic-operated platforms — the prompt cache.
`workspace:developer`	The OAuth scope matching the access an API key issued for the same workspace has.

26.41 Further Reading

Primary sources, all verified on 1 August 2026 at `platform.claude.com/docs`:

- `api/overview` — endpoints, the header table, request-size limits, response headers and pagination.
- `manage-claude/authentication` — the two methods, key expiration and the case for federation.
- `manage-claude/workload-identity-federation` and `manage-claude/wif-reference` — the exchange, environment variables, **credential precedence**, profile files, scopes, validation rules and errors.
- `api/versioning` — what a version preserves and what it permits to change.
- `api/beta-headers` — the header, multiple betas, endpoint-scoped headers, naming conventions and the 400.
- `cli-sdks-libraries/overview` — the seven SDKs, the CLI and the libraries.
- `cli-sdks-libraries/middleware` — registration, ordering, the HTTP client, and the built-in refusal-fallback middleware.
- `cli-sdks-libraries/cli/using` and `.../cli/authentication` — command structure, formats, GJSON transforms, request bodies, `--debug`, profiles and `ant auth status`.

- `cli-sdks-libraries/sdks/python` — retries, timeouts, long requests, auto-pagination, default headers and platform clients.
- `manage-claude/admin-api` and `manage-claude/workspaces` — roles, scoping, key management and workspace limits.
- `api/errors` — status codes, the error shape, typed SDK exceptions and request IDs.

Where to go next in this book. Chapter 27 takes the body apart parameter by parameter, including the sampling controls this chapter deliberately left alone. **Chapter 28** returns to `stop_reason` and response handling in depth. **Chapter 29** turns §26.22's arithmetic into a retry and backoff discipline, with the error taxonomy of §26.13 as its input. **Chapter 30** covers batch, prompt caching and cost controls, where §26.3's per-workspace cache scope becomes a design decision. Further out, **Chapter 45** revisits permissions and safety for Claude Code, and **Chapter 58** treats least privilege — the principle behind §26.23's blast-radius table — as a security discipline in its own right.